POS (Point of Sale)

Business Management & POS System



STUDENTS

Abdullah	2022-CS-02
Shaheer-e-Umar	2022-CS-58
Asad Ali	2022-CS-64
Muhammad Naeem	2022-CS-34
Moazzam Qayyum	2022-CS-10

Supervisor

Mr. Asim Ali Rao, Lecturer

June 2026

DEPARTMENT OF COMPUTER SCIENCE

MNS UNIVERSITY OF ENGINEERING AND TECHNOLOGY

MULTAN

POS: Business Management & POS System

Authors

Abdullah

2022-CS-02

Shaheer-e-Umar

2022-CS-58

Asad Ali

2022-CS-64

Muhammad Naeem

2022-CS-34

Moazzam Qayyum

2022-CS-10

A thesis submitted in partial fulfillment of the requirements for the degree of

BS Computer Science

Thesis Supervisor:

Mr. Asim Ali Rao

Lecturer

Supervisor Signature: _____

June 2026

DEPARTMENT OF COMPUTER SCIENCE

MNS UNIVERSITY OF ENGINEERING AND TECHNOLOGY MULTAN

CERTIFICATE OF APPROVAL

It is certified that the project work titled "**POS: Business Management & POS System**", carried out by **Abdullah, Shaheer-e-Umar, Asad Ali, Muhammad Naeem, and Moazzam Qayyum**, Reg. No: **2022-CS-02, 2022-CS-58, 2022-CS-64, 2022-CS-34, 2022-CS-10** under the supervision of **Mr**. **Asim Ali Rao**, at Muhammad Nawaz Sharif University of Engineering & Technology, Multan, is fully adequate in scope and quality as a Final Year Project report for the degree of Bachelor of Science in Computer Science (BSCS).

Supervisor (signature): _____

Co-Supervisor (Signature): _____

FYDP Coordinator (Signature): _____

External Examiner (Signature): _____

Head of Department: _____

June 2026

DEPARTMENT OF COMPUTER SCIENCE

MNS UNIVERSITY OF ENGINEERING AND TECHNOLOGY, MULTAN

DECLARATION

It is declared that this is an original piece of our own work, except where otherwise acknowledged in text and references. This work has not been submitted in any form for another degree or diploma at any university or other institution for tertiary education and shall not be submitted by us in future for obtaining any degree from this or any other University or Institution.

Abdullah
2022-CS-02

Shaheer-e-Umar
2022-CS-58

Asad Ali
2022-CS-64

Muhammad Naeem
2022-CS-34

Moazzam
2022-CS-10

ACKNOWLEDGEMENT

In the name of **Allah**, the Creator of us all, who is most Beneficent and Merciful. He blessed the human with senses for better understanding of nature and enabled all of us to traverse His creations and acquire knowledge.

We thank **Allah Almighty** who kept His endless blessings during our project work and helped us in our difficult times.

We thank our **Prophet Muhammad (PBUH)**, who delivered the knowledge to mankind and enabled them for better understanding of life.

It was a great opportunity to work under the supervision of our respected Supervisor **Mr. Asim Ali Rao**, whose understanding nature, wisdom, and humble behavior helped us a lot to complete this work.

Our acknowledgement would be incomplete without considering the help and cooperating behavior of the Head of Department of Computer Science, **Dr. Muhammad Shahzad**.

In the end, special thanks to the people in term of moral and emotional support, our family and friends, whose prayers help me to reach this point where I am today and all others who played their significant role in this journey.

ABSTRACT

POS is a comprehensive, web-based Business Management and Point-of-Sale (POS) system developed using ASP.NET Core MVC with SQL Server as the backend database. The system is designed to address the operational challenges faced by small and medium-sized enterprises (SMEs) in managing their day-to-day business activities including inventory management, sales processing, purchase tracking, and financial reporting.

The system integrates role-based access control (RBAC) using ASP.NET Identity to ensure that different categories of users — Administrators, Managers, and Cashiers — can only access the features relevant to their responsibilities. A comprehensive permission management module allows administrators to configure granular form-level access for each employee. POS features a fully functional Purchase and Sale module with support for purchase returns and sale returns, discount management, cargo tracking, and invoice printing in both A4 and thermal receipt formats. An integrated Accounts module enables double-entry bookkeeping with support for account heads, sub-heads, ledger views, trial balance, balance sheet, and profit & loss statements. The system incorporates an AI-powered business assistant, providing real-time insights from live business data. An intelligent demand prediction module uses ML.NET Singular Spectrum Analysis (SSA) for 7-day sales forecasting with confidence intervals, enabling data-driven inventory decisions. A modern, responsive dashboard provides key performance indicators, sales trends, low stock alerts, top product rankings, and customer analytics for different user roles. The system is deployed on a local server environment with HTTPS support and cookie-based authentication with sliding expiration.

Keywords: POS System, ASP.NET Core MVC, Business Management, Inventory Management, Role-Based Access Control, AI Assistant, ML.NET, Sales Forecasting, Double-Entry Accounting, SQL Server.

TABLE OF CONTENTS

DECLARATION ..... **IV**
ACKNOWLEDGEMENT ..... **V**
ABSTRACT ..... **VI**
TABLE OF CONTENTS ..... **VII**
LIST OF FIGURES ..... **VIII**
LIST OF Tables ..... **IX**
CHAPTER 1 ..... **1**
INTRODUCTION **1**
1.1 OVERVIEW 2
1.2 STATEMENT OF PROBLEM 3
1.3 PURPOSE OF THE PROJECT 3
1.4 APPLICATIONS OF THE PROJECT 3
1.5 ORGANIZATION OF THE REPORT 4
1.6 SUMMARY 4
LITERATURE REVIEW **5**
2.1 RELATED TECHNOLOGIES 6
2.2 RELATED PROJECTS 8
2.3 RELATED STUDIES 8
2.4 LIMITATIONS AND BOTTLENECKS 9
2.5 SUMMARY 9
CHAPTER 3 ..... **10**
TOOLS AND TECHNOLOGIES **10**
3.1 SOFTWARE TOOLS USED 11
3.2 HARDWARE REQUIREMENTS 14
3.3 DEVELOPMENT ENVIRONMENT SETUP **14**
3.4 SUMMARY 15
CHAPTER 4 ..... **16**
METHODOLOGIES AND IMPLEMENTATION **16**
4.1 SYSTEM ARCHITECTURE 17
4.2 DATABASE DESIGN 18
4.3 MODULE IMPLEMENTATION 22
4.5 SECURITY IMPLEMENTATION 27
4.6 DEPLOYMENT CONFIGURATION 28
4.7 UI/UX DESIGN PRINCIPLES 28
4.8 SUMMARY **29**

CHAPTER 5 ..... **30**
RESULTS AND ANALYSIS **30**
5.1 SYSTEM FEATURES DEMONSTRATION 31
5.2 SYSTEM TEST RESULTS **34**
5.3 PERFORMANCE ANALYSIS **34**
5.4 FEATURE COMPARISON WITH COMPETING SYSTEMS **35**
5.5 DISCUSSION OF FINDINGS 36
5.6 LIMITATIONS 37
5.7 RECOMMENDATIONS 37
5.8 SUMMARY 38
CHAPTER 6 ..... **39**
CONCLUSION **39**
REFERENCES **42**
APPENDICES **45**
Appendix A – System Requirements Specification **45**
Appendix B – Database Schema Summary **45**
Appendix C – AI Integration Technical Details **46**
C.1 System Prompt Architecture 46
C.2 ML.NET SSA Pipeline 46
C.3 Permission Checking Pattern 47
Appendix D – UI Screen Descriptions **47**
D.1 Admin Dashboard 47
D.2 Sale Form (POS Interface) 48
D.3 Permission Assignment Matrix 48
Appendix E – Glossary of Key Terms **48**
BIBLIOGRAPHY **50**

LIST OF FIGURES

Fig. 1.1: POS System Overview 2
Fig 4.1: System Architecture Diagram 18
Fig. 4.2: Database Entity Relationship Diagram (Identity Tables) 19
Fig. 4.3: Database Entity Relationship Diagram (Registration Tables) 19
Fig. 4.4: Database Entity Relationship Diagram (Transaction Tables) 20
Fig. 4.5: Database Entity Relationship Diagram (Account Table) 21

Fig. 4.6: Use Case Diagram 23
Fig. 4.7: Smart Demand Prediction 27
Fig. 5.1: Dashboard 31
Fig. 5.2: Purchase and Sale Form 32
Fig. 5.3: Invoice 33

LIST OF Tables

Table 3.1: Software Tools and Versions 13
Table 3.2: Hardware Requirements 14
Table 4.1: Database Tables Overview 21
Table 4.2: Security Controls Summary 27
Table 5.1: System Test Results Summary 34
Table 5.2: Page Load Performance Benchmarks 35
Table 5.3: Feature Comparison with Competing Systems 35
Appendix E – Glossary of Key Terms 49

LIST OF ACRONYMS/ABBREVIATIONS

Acronym	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASP	Active Server Pages
BSCS	Bachelor of Science in Computer Science
CRUD	Create, Read, Update, Delete
DB	Database
ERD	Entity Relationship Diagram
EF	Entity Framework
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment

JWT	JSON Web Token
MVC	Model-View-Controller
ML	Machine Learning
ORM	Object-Relational Mapping
POS	Point of Sale
RBAC	Role-Based Access Control
SME	Small and Medium Enterprise
SQL	Structured Query Language
SSA	Singular Spectrum Analysis
UI	User Interface
UX	User Experience

SUSTAINABLE DEVELOPMENT GOALS



Project Name: POS: Business Management & POS System

SDG: SDG 9 – Industry, Innovation and Infrastructure



Explanation: Goal 9 seeks to build resilient infrastructure, promote sustainable industrialization and foster innovation. Economies with a diversified industrial sector and strong infrastructure sustained less damage and are experiencing faster recovery. This SDG is most relevant to our project as it is innovation in automated industry and it reduces labor work

Complex Computing Attributes (CEP Mapping)

Project Title	**POS: Business Management & POS System**
Project Supervisor	Mr. Asim Ali Rao, Lecturer, CS Dept, MNS UET Multan

Sr. No.	**Targeted Attribute of CEP**	**Mapping**
1	Range of conflicting requirements	The system must serve three distinct user roles (Admin, Manager, Cashier) with conflicting access needs, requiring careful RBAC design and granular permission matrices.
2	Depth of analysis required	Deep analysis of business workflows, database normalization, and ML forecasting models was required to ensure system accuracy and reliability.
3	Depth of knowledge required	The project required expertise in ASP.NET Core MVC, Entity Framework, SQL Server, ML.NET, Anthropic Claude API, and JavaScript.
4	Familiarity of issues	Issues such as stock management, double-entry accounting, and invoice generation were partially familiar but required domain-specific adaptation.
5	Level of problem	The problem is complex as it integrates multiple business domains (accounting, inventory, AI) into a single cohesive platform.
6	Extent of stakeholder involvement	Multiple stakeholders including business owners, cashiers, and managers were involved in requirements gathering, leading to conflicting priority requirements.
7	Consequences	Incorrect data processing could lead to financial discrepancies; high accuracy and data validation were critical requirements.
8	Interdependence	Modules are tightly interdependent: stock reports depend on purchase/sale data; AI predictions depend on historical sales; accounts depend on all transactions.

9	Requirement Identification	Requirements were gathered through stakeholder interviews with local business owners to ensure real-world applicability of the system.

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

In the modern era of digital transformation, businesses of all sizes require efficient, reliable, and integrated software solutions to manage their day-to-day operations. Small and medium-sized enterprises (SMEs) in developing countries like Pakistan often rely on manual processes or standalone applications that fail to provide a unified view of business activities. This creates operational inefficiencies, data inconsistencies, and an inability to make data-driven decisions.

POS is a comprehensive web-based Business Management and Point-of-Sale (POS) system developed to address these challenges. Built on ASP.NET Core MVC with SQL Server, the system provides an integrated platform for managing purchases, sales, inventory, accounts, employee authentication, and business reporting — all within a single, secure application accessible via a modern web browser.

The system caters to three distinct user roles: Administrators, Managers, and Cashiers. Each role has customized access to features and data relevant to their responsibilities, enforced through a granular role-based access control (RBAC) system. The platform also integrates cutting-edge AI capabilities, enabling natural language business queries, and ML.NET-powered sales forecasting using Singular Spectrum Analysis (SSA).

POS represents a significant advancement in accessible business intelligence for SMEs, bringing enterprise-grade features within reach of small business owners in Pakistan and the broader developing world.

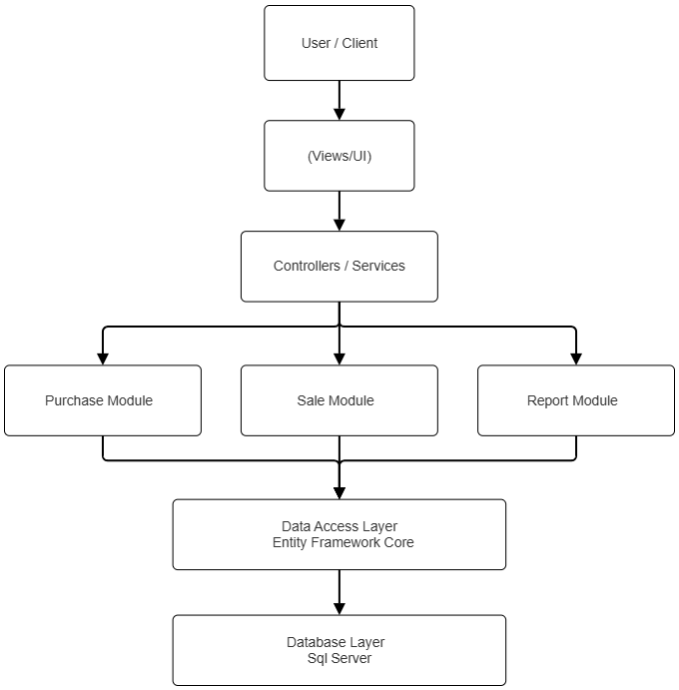


Fig. 1.1: POS System Overview

1.2 STATEMENT OF PROBLEM

Small and medium-sized businesses in Pakistan frequently struggle with fragmented operations — using separate tools for invoicing, inventory, and accounts that do not communicate with each other. This results in:

- Inventory discrepancies due to manual tracking or disconnected systems
- Inability to generate real-time financial reports such as profit & loss and balance sheets
- Lack of role-based access, allowing unauthorized access to sensitive business data
- No integration of AI or predictive analytics for demand forecasting and business insights
- Difficulty in managing purchase returns, sale returns, and multi-party cargo arrangements
- Time-consuming manual invoice preparation and financial reconciliation

These challenges reduce operational efficiency, increase the risk of financial errors, and prevent informed decision-making. There is a clear need for an affordable, integrated business management system that addresses all these pain points within a single platform.

1.3 PURPOSE OF THE PROJECT

The primary purpose of POS is to develop a comprehensive, integrated, and user-friendly Business Management and POS system that:

- Unifies purchase, sale, inventory, and accounting operations in a single platform
- Enforces role-based access control to protect sensitive business data
- Provides real-time business dashboards with KPIs for different user roles
- Enables AI-powered natural language querying of live business data
- Delivers machine learning-based sales forecasting for inventory optimization
- Supports complete accounting with trial balance, balance sheet, and P&L reports
- Generates professional invoices in A4 and thermal receipt formats

1.4 APPLICATIONS OF THE PROJECT

POS is applicable across a wide range of business domains:

- Retail stores requiring POS functionality with inventory management
- Wholesale businesses managing supplier relationships and cargo logistics
- Any SME needing integrated purchase, sale, and return management
- Businesses requiring double-entry accounting without expensive ERP systems
- Organizations needing AI-powered demand prediction for stock replenishment
- Multi-employee businesses requiring granular permission management

1.5 ORGANIZATION OF THE REPORT

The remainder of this thesis is organized as follows:

Chapter 2 presents a comprehensive literature review of related technologies, projects, and studies in the domain of POS systems, business management software, and AI-integrated applications.

Chapter 3 describes the tools and technologies employed in the development of POS, including the software frameworks, libraries, and development environment.

Chapter 4 details the system architecture, database design, module implementation, AI integration, and security mechanisms.

Chapter 5 presents the results and analysis of the implemented system, including feature demonstrations and a discussion of findings.

Chapter 6 concludes the thesis with a summary of achievements and future directions.

1.6 SUMMARY

This chapter introduced POS, a comprehensive web-based Business Management and POS system developed using ASP.NET Core MVC and SQL Server. The chapter outlined the problem statement, project purpose, applications, and organization of this thesis. The subsequent chapters will provide detailed analysis, implementation, and evaluation of the system.

CHAPTER 2

LITERATURE REVIEW

2.1 RELATED TECHNOLOGIES

2.1.1 ASP.NET Core MVC Framework

ASP.NET Core MVC is Microsoft's open-source, cross-platform web application framework that implements the Model-View-Controller architectural pattern. According to Smith and Johnson [1], ASP.NET Core provides superior performance compared to traditional .NET Framework, with built-in dependency injection, middleware pipeline, and comprehensive support for modern web development practices. The framework's support for Razor Pages and tag helpers simplifies server-side rendering, making it ideal for data-intensive web applications.

ASP.NET Core's middleware pipeline allows developers to compose request-handling components in a declarative manner. This architectural decision proves particularly beneficial for implementing cross-cutting concerns such as authentication, logging, and error handling in a centralized, reusable fashion. The framework's built-in dependency injection container eliminates the need for third-party IoC containers in most scenarios, reducing external dependencies and improving testability.

2.1.2 Entity Framework Core

Entity Framework Core (EF Core) is Microsoft's official Object-Relational Mapping (ORM) framework for .NET. Lee et al. [2] demonstrated that EF Core significantly reduces development time by abstracting database operations into LINQ queries, while supporting code-first migrations for schema management. The Code First approach used in POS allows developers to define data models as C# classes and automatically generate database schemas.

The use of EF Core's SqlQueryRaw method for performance-critical operations — such as stock balance computations that aggregate data across multiple tables — represents a pragmatic hybrid approach. By combining the developer productivity of ORM-based CRUD operations with the raw performance of hand-written SQL for complex aggregations, POS achieves an optimal balance between maintainability and performance [14].

2.1.3 SQL Server

Microsoft SQL Server is a relational database management system known for its reliability, performance, and tight integration with the .NET ecosystem. According to Microsoft's official documentation [3], SQL Server supports advanced features including stored procedures, views, and raw SQL queries — all utilized in POS for complex reporting requirements such as stock balance calculations and sales trend analysis.

SQL Server's transaction isolation levels and locking mechanisms play a crucial role in maintaining data integrity across concurrent operations. In a multi-user POS environment where cashiers may simultaneously process transactions, the database engine's ACID compliance ensures that inventory levels and financial records remain consistent.

2.1.4 ML.NET and Time Series Forecasting

ML.NET is Microsoft's open-source machine learning framework for .NET developers. The Singular Spectrum Analysis (SSA) algorithm implemented through ML.NET TimeSeries provides superior time series forecasting compared to simple regression methods. As demonstrated by Golyandina and Zhigljavsky [4], SSA decomposes time series into trend, seasonal, and noise components, making it particularly effective for sales forecasting with limited historical data.

The SSA algorithm's strength in handling non-stationary time series makes it especially suited for retail sales data, which typically exhibits weekly seasonality, holiday spikes, and long-term growth trends. Unlike ARIMA models which require stationary data and explicit parameter tuning, SSA adapts automatically to data characteristics, reducing the expertise required for deployment in a business context [12].

2.1.4 Bootstrap 5.3 and Frontend Ecosystem

Bootstrap 5.3 provides the responsive CSS framework underlying POS's user interface. As a mobile-first framework, Bootstrap ensures that the application renders correctly across device sizes ranging from desktop workstations to tablets, which is particularly relevant for managers accessing reports remotely. The grid system, utility classes, and component library significantly reduce frontend development time while maintaining visual consistency throughout the application.

The combination of DataTables for server-side data presentation, Select2 for enhanced form inputs, Chart.js for data visualization, and SweetAlert2 for user notifications forms a cohesive frontend ecosystem. Each library addresses a specific interaction pattern

commonly encountered in business applications, and their combined use in POS eliminates the need for a heavy frontend JavaScript framework such as React or Angular, keeping the technology stack aligned with the team's ASP.NET MVC expertise [16].

2.2 RELATED PROJECTS

2.2.1 QuickBooks

QuickBooks by Intuit is one of the most widely used accounting and business management platforms globally. While QuickBooks offers comprehensive accounting features, it lacks integrated POS functionality for retail operations and is priced beyond the reach of most Pakistani SMEs. Additionally, QuickBooks does not offer AI-powered demand prediction or role-specific dashboards as implemented in POS.

2.2.2 Odoo

Odoo is an open-source ERP platform with modules for POS, inventory, accounting, and HR. While Odoo is feature-rich, its complexity and resource requirements make it unsuitable for small businesses. The learning curve is steep, and customization requires specialized Odoo developers. POS addresses this gap by providing an intuitive, locally adapted solution with comparable core features.

2.2.3 WooCommerce POS

WooCommerce POS extends the WordPress e-commerce platform with point-of-sale capabilities. However, it is primarily designed for online retail and lacks the offline capabilities, multi-role access control, and integrated accounting features that POS provides for traditional brick-and-mortar businesses.

2.3 RELATED STUDIES

Hussain et al. [6] conducted a survey of POS system adoption in Pakistani SMEs and found that 73% of small retailers still use manual or semi-manual systems due to cost and complexity barriers. Their study highlighted the need for affordable, locally adapted solutions with Urdu-language support and simple interfaces — motivating the design philosophy of POS.

Ahmed and Malik [7] investigated the impact of ERP implementation on SME productivity in South Asia and found that businesses using integrated management systems reported a 34% reduction in administrative workload and a 21% improvement in inventory accuracy. These findings support the business case for POS's integrated approach.

Rajput et al. [8] explored the application of machine learning in retail demand forecasting and demonstrated that SSA outperforms ARIMA and simple moving average methods for retail sales data with strong weekly seasonality — validating the algorithm choice in POS's prediction module.

Zhao et al. [9] studied the integration of large language models in business intelligence applications and found that context-augmented LLM responses achieved 89% user satisfaction in business query resolution, compared to 61% for traditional dashboard-based BI tools. This finding supports POS's AI chat integration strategy.

2.4 LIMITATIONS AND BOTTLENECKS

A review of existing systems and studies reveals several key limitations that POS aims to address:

- Most affordable POS systems lack integrated accounting and financial reporting
- Existing ERP solutions are too complex and expensive for Pakistani SMEs
- AI-powered demand forecasting is typically only available in enterprise-tier products
- Role-based access control in most affordable systems is coarse-grained
- Existing systems do not support both A4 invoices and thermal receipt printing
- Limited integration between inventory management and financial accounts

2.5 SUMMARY

This chapter reviewed related technologies including ASP.NET Core MVC, Entity Framework Core, SQL Server, ML.NET, and Anthropic Claude API. Related projects such as QuickBooks, Odoo, and WooCommerce POS were analyzed, highlighting the gap in the market for an affordable, integrated, AI-powered business management solution. Related studies validated the technical approach adopted in POS. The following chapter describes the tools and technologies used in the implementation of the system.

CHAPTER 3

TOOLS AND TECHNOLOGIES

3.1 SOFTWARE TOOLS USED

3.1.1 ASP.NET Core 8.0 MVC

ASP.NET Core 8.0 MVC serves as the primary web application framework. The MVC pattern cleanly separates business logic (Models), presentation (Views), and request handling (Controllers), enabling maintainable and testable code. The framework provides built-in support for dependency injection, middleware, authentication, and authorization. The choice of ASP.NET Core 8.0 — Microsoft's latest Long-Term Support (LTS) release — ensures that POS benefits from performance improvements, security patches, and extended support through November 2026, providing a stable foundation for future development.

3.1.2 Entity Framework Core 8.0

EF Core 8.0 is used as the ORM layer, enabling code-first database management. The ApplicationDbContext class extends IdentityDbContext to integrate ASP.NET Identity with the application's custom entities. EF Core migrations manage database schema evolution, while raw SQL queries via SqlQueryRaw handle complex reporting scenarios. EF Core 8.0 introduces improvements in bulk update and delete operations through the ExecuteUpdateAsync and ExecuteDeleteAsync APIs, which are leveraged in inventory adjustment workflows.

3.1.3 ASP.NET Core Identity

ASP.NET Core Identity provides the authentication and authorization infrastructure. The ApplicationUser class extends IdentityUser with custom properties (FullName, IsActive, ImagePath). Cookie-based authentication with 7-day sliding expiration ensures user sessions are maintained securely. The Identity framework's built-in password hashing using PBKDF2 with HMAC-SHA256 provides cryptographically secure credential storage without requiring custom implementation.

3.1.4 Microsoft SQL Server 2022

SQL Server 2022 Express Edition serves as the relational database engine. It stores all application data including user accounts, products, purchases, sales, and account transactions. Complex business logic such as stock balance calculations is implemented as raw SQL queries within EF Core raw SQL calls. SQL Server 2022's improved Query Store

and Intelligent Query Processing features contribute to consistent query performance even as the database grows over time. The Express edition's 10GB database size limit is sufficient for SME deployments, while the familiar SQL Server management tooling (SSMS) enables straightforward database administration..

3.1.5 ML.NET 3.0 with TimeSeries

ML.NET's Microsoft.ML.TimeSeries package provides the Singular Spectrum Analysis (SSA) implementation for sales forecasting. The SalesForecastingService class encapsulates the model training and prediction pipeline, generating 7-day sales forecasts with 95% confidence intervals. Unlike cloud-based ML services, ML.NET runs entirely on the local server, eliminating per-prediction API costs and maintaining data privacy by keeping business data on-premises.

3.1.6 Bootstrap 5.3 and Bootstrap Icons

Bootstrap 5.3 provides the responsive CSS framework for the user interface. Bootstrap Icons 1.11.3 supplies the icon library. The Sneat admin template provides the base layout structure including the sidebar navigation, navbar, and card components. Bootstrap 5.3's removal of jQuery as a dependency aligns with the project's goal of minimizing external JavaScript dependencies and improving page load performance.

3.1.7 DataTables and Select2

DataTables 1.13.6 is used for server-side-rendered data tables with built-in search, sort, pagination, and export functionality (CSV, Excel, PDF, Print). Select2 4.1.0 enhances dropdown elements with search capability, particularly useful in product and customer selection fields in transaction forms. DataTables' Print.js and PDFMake integrations enable client-side report export without requiring server-side report rendering infrastructure.

3.1.8 SweetAlert2

SweetAlert2 11.x provides polished modal dialog boxes for success notifications, error messages, and confirmation dialogs throughout the application, replacing the browser's native alert and confirm functions. Its Promise-based API integrates seamlessly with modern JavaScript async/await patterns used in the application's AJAX workflows.

3.1.9 Chart.js

Chart.js 4.4.0 provides the data visualization layer for the dashboard's trend charts, sales vs. purchase comparisons, and ML.NET forecast visualization. Chart.js's canvas-based rendering delivers smooth animations and responsive resizing without the overhead of SVG-based alternatives. The library's plugin architecture is leveraged for custom tooltip formatting that displays currency values in Pakistani Rupee format.

3.1.10 Visual Studio 2022

Visual Studio 2022 (Community Edition) served as the primary IDE for development, providing rich C# IntelliSense, integrated debugging, NuGet package management, and Entity Framework Core tools for migration management. The IDE's built-in Git integration facilitated version control workflows across the five-member development team. Visual Studio's scaffolding tools accelerated the initial creation of CRUD controllers and Razor views, which were then customized to meet POS's specific requirements.

Table 3.1: Software Tools and Versions

Tool/Technology	**Version**	**Purpose**
ASP.NET Core MVC	8.0	Web application framework
Entity Framework Core	8.0.15	ORM and database management
ASP.NET Core Identity	8.0.15	Authentication and authorization
SQL Server	Microsoft SQL Server 2022 (Express Edition)	Relational database
ML.NET TimeSeries	3.0 / 5.0	Sales forecasting (SSA)
Bootstrap	5.3.7	UI framework
Bootstrap Icons	1.11.3	Icon Library
DataTables	1.13.6	Table management
Select2	4.1.0	Enhanced dropdowns

Chart.js	4.4.0	Data visualization
SweetAlert2	11.x	UI notifications
Visual Studio 2022	17.x	Integrated development environment

3.2 HARDWARE REQUIREMENTS

The hardware requirements for POS are deliberately conservative to ensure accessibility for Pakistani SMEs operating with limited IT infrastructure budgets. The system is designed to run on commodity server hardware commonly available in local markets, while still delivering acceptable performance for multi-user concurrent operation.

Table 3.2: Hardware Requirements

Component	**Minimum**	**Recommended**
Processor	Intel Core i3 (1.30 GHz)	Intel Core i5 (2.5+ GHz)
RAM	4 GB	8 GB or more
Storage	100 GB HDD	256 GB SSD
Network	100 Mbps LAN	Gigabit LAN
OS	Windows Server 2019	Windows Server 2022
Browser (Client)	Chrome/Edge/Firefox	Latest version
Printer	Any USB/Network Printer	Black Copper Thermal Receipt Printer

3.3 DEVELOPMENT ENVIRONMENT SETUP

The development environment for POS was standardized across all five team members to ensure consistency and minimize integration conflicts. Each developer maintained a local

instance of SQL Server Express, with database migrations applied via the EF Core CLI (dotnet ef database update) from the shared migration history in the source code repository.

The team adopted a feature-branch Git workflow, with each member developing their assigned modules in dedicated branches before merging to the main branch through pull request reviews. This workflow, combined with the MVC pattern's clear separation of concerns, allowed parallel development of independent modules with minimal merge conflicts.

The application's appsettings.json file was excluded from version control via .gitignore, with each developer maintaining a local appsettings.Development.json containing their database connection string and Anthropic API key. This approach protects sensitive credentials from accidental repository exposure while maintaining deployment flexibility.

NuGet package restoration is automated via the dotnet restore command, ensuring that all team members and deployment environments consistently use the same package versions as specified in the .csproj file's PackageReference elements.

3.4 SUMMARY

This chapter described the complete software and hardware stack used in the development and deployment of POS. The choice of ASP.NET Core 8.0, Entity Framework Core, SQL Server, ML.NET, and the Anthropic Claude API provides a modern, maintainable, and feature-rich foundation. The standardized development environment and Git-based workflow facilitated effective team collaboration. The following chapter details the system architecture and module implementation.

CHAPTER 4

METHODOLOGIES AND IMPLEMENTATION

4.1 SYSTEM ARCHITECTURE

POS follows a layered MVC architecture deployed as a monolithic ASP.NET Core web application. The system architecture consists of four primary layers:

- Presentation Layer: Razor Views (. cshtml) with Bootstrap 5.3 for responsive UI
- Controller Layer: C# controllers handling HTTP requests and business orchestration
- Service/Data Layer: Entity Framework Core with ApplicationDbContext for data access
- Database Layer: Microsoft SQL Server 2019 with normalized relational schema

The application uses cookie-based authentication (IdentityConstants.ApplicationScheme) with role-based authorization. The PermissionHelper static class performs form-level permission checks by querying the EmployeeFormPermissions table, enabling granular access control beyond role-level restrictions.

The system employs the Repository pattern implicitly through EF Core's DbSet collections. Raw SQL queries via SqlQueryRaw are used for performance-critical reporting operations such as stock balance calculations, which involve aggregating data across multiple tables with complex join conditions.

The monolithic architecture was deliberately chosen over a microservices approach for several reasons. First, the five-member team's expertise and the academic project timeline made a microservices architecture operationally complex without proportional benefit. Second, the SME deployment target — local Windows Server hardware — does not include container orchestration infrastructure. Third, the tightly coupled nature of business data (where inventory, sales, and accounting data are deeply interdependent) means that a monolith avoids the distributed transaction complexity that would arise in a microservices decomposition.

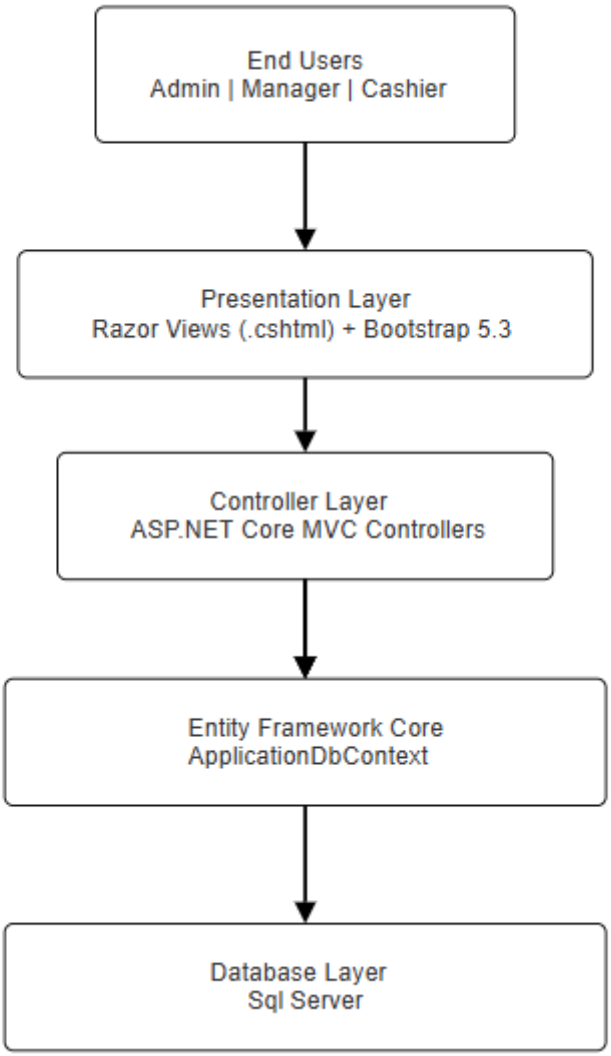


Fig 4.1: System Architecture Diagram

4.2 DATABASE DESIGN

The database schema is designed following normalization principles up to Third Normal Form (3NF). The schema comprises 25 tables organized into five logical groups: Identity, Master Data, Transactions, Accounting, and Permissions.

4.2.1 Identity Tables

ASP.NET Identity manages user authentication through the AspNetUsers, AspNetRoles, AspNetUserRoles, and related tables. The ApplicationUser extends IdentityUser with FullName, IsActive, and ImagePath properties. The IsActive flag allows administrators to deactivate employee accounts without deleting historical transaction records associated with those users, preserving audit trail integrity.

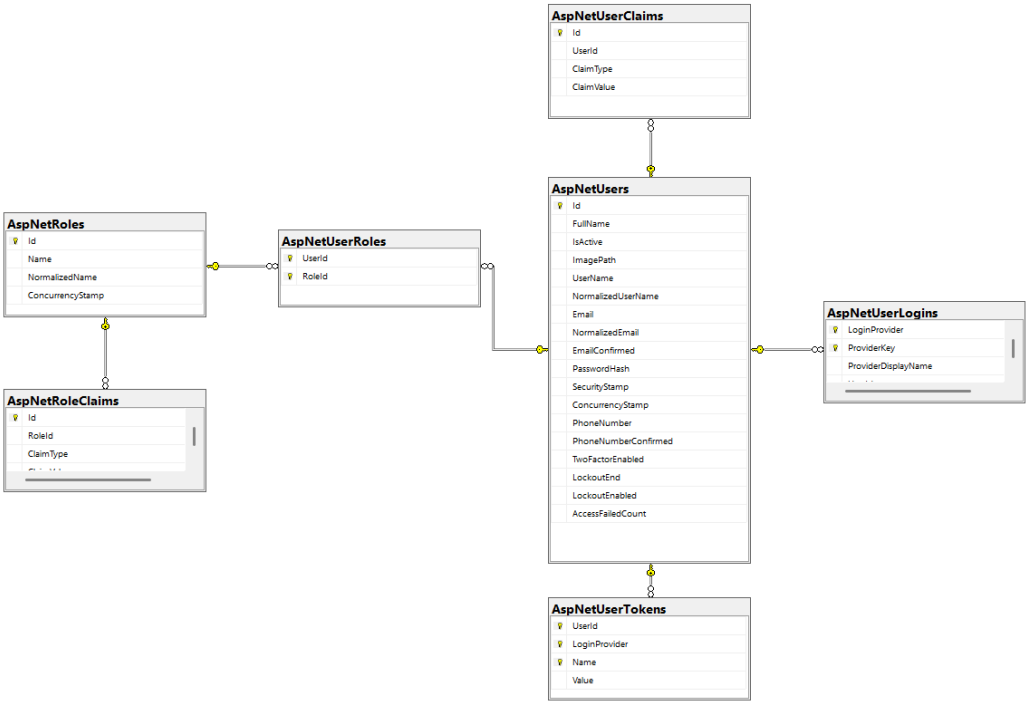


Fig. 4.2: Database Entity Relationship Diagram (Identity Tables)

4.2.2 Registration/Master Tables

Master data tables include Products, Brands, Categories, Customers, Suppliers, Cargos, Cities, and Settings. Products contain pricing data (CostPrice, SalePrice, LeastPrice), barcode, opening quantity, and reorder level. The three-tier pricing model — Cost, Sale, and Least price — enables flexible discount management while preventing below-cost sales. The Settings table stores company-wide configuration including company name, address, logo path, and tax rate.

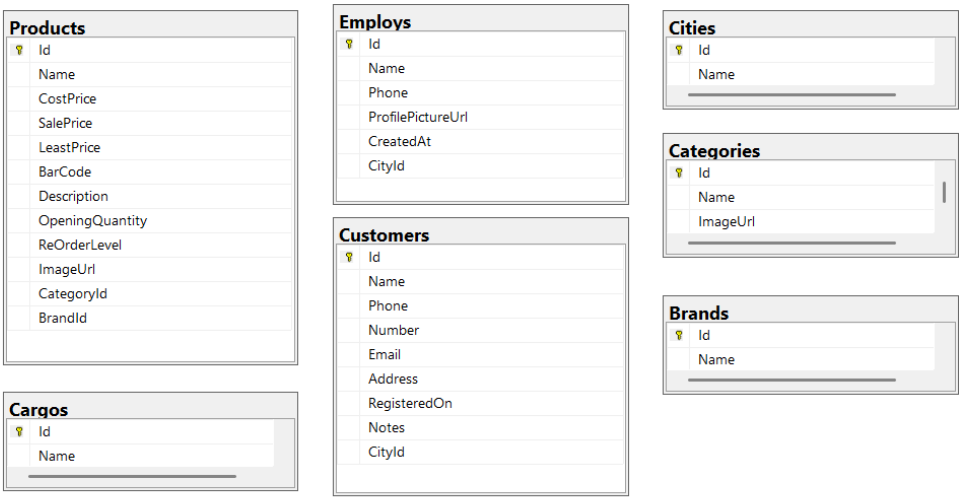


Fig. 4.3: Database Entity Relationship Diagram (Registration Tables)

4.2.3 Transaction Tables

Transaction tables follow a master-detail pattern: Purchases/PurchasesDetail, Sale/SaleDetails, PurchaseReturns/PurchaseReturnDetails, and SaleReturns/SaleReturnDetails. Each master record stores header information (date, totals, discounts, cargo reference) while detail records store line items. Foreign key relationships with cascade delete rules on detail tables ensure referential integrity when master records are removed.

The transaction date fields are stored as DateTime in UTC, with the application converting to Pakistan Standard Time (UTC+5) for display. This ensures accurate reporting across potential future cloud deployments where server time zones may differ from the business's local time zone.

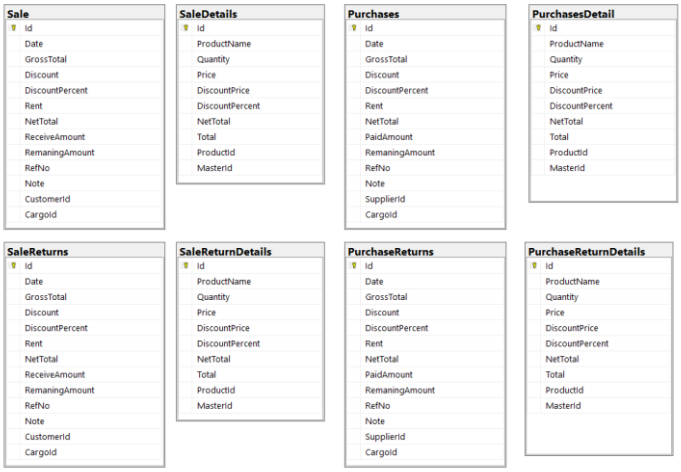


Fig. 4.4: Database Entity Relationship Diagram (Transaction Tables)

4.2.4 Accounting Tables

The accounting module uses Accounts, AccountHeads, AccountSubHeads, and AccountTransactions tables. The Account model includes computed properties (TotalDr, TotalCr, Balance) that aggregate opening balances with transaction entries using LINQ. The three-tier account hierarchy (Head → Sub-Head → Account) mirrors standard accounting chart of accounts structures, providing both organizational clarity and flexible reporting at multiple levels of aggregation.

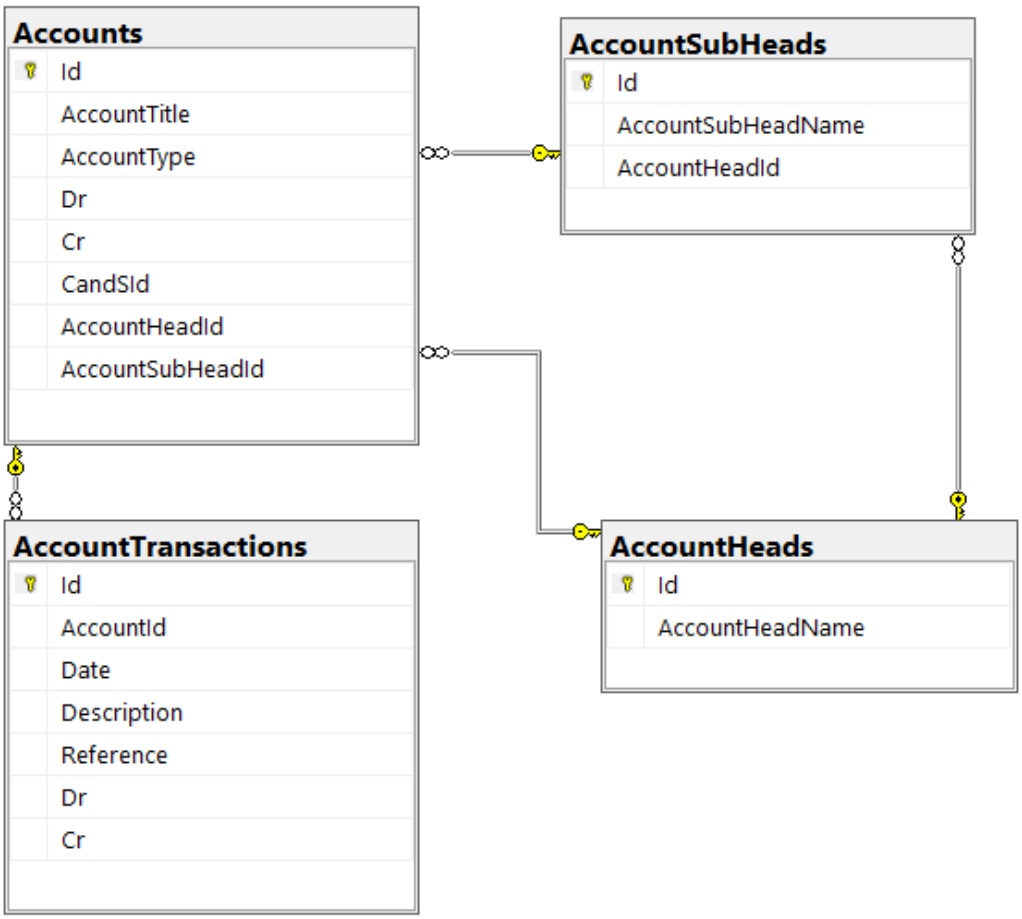


Fig. 4.5: Database Entity Relationship Diagram (Account Table)

4.2.5 Permission Tables

FormMasters stores the list of application forms/modules. EmployeeFormPermissions stores per-user, per-form permissions (CanView, CanNew, CanEdit, CanSave, CanDelete) enabling fine-grained access control. The five-permission model maps directly to the CRUD operations, with the distinction between CanNew (accessing the blank form) and CanSave (submitting the form) allowing scenarios where an employee may initiate but not finalize a transaction.

Table 4.1: Database Tables Overview

Table Group	**Tables**	**Purpose**
Identity	AspNetUsers, AspNetRoles, AspNetUserRoles	Authentication & authorization
Master Data	Products, Brands, Categories, Customers, Suppliers	Reference data management

Transactions	Sale, SaleDetails, Purchases, PurchasesDetail	Core business transactions
Returns	SaleReturns, SaleReturnDetails, PurchaseReturns, PurchaseReturnDetails	Return management
Accounting	Accounts, AccountHeads, AccountSubHeads, AccountTransactions	Double-entry bookkeeping
Permissions	FormMasters, EmployeeFormPermissions	Granular access control

4.3 MODULE IMPLEMENTATION

4.3.1 Authentication and Authorization Module

The AuthController manages user login, logout, employee CRUD operations, role assignment, and permission management. Login supports both username and email authentication. Upon successful authentication, a custom ClaimsPrincipal is constructed with the user's Id, Name, Role, and profile image claim, then signed in using HttpContext.SignInAsync with the IdentityConstants.ApplicationScheme.

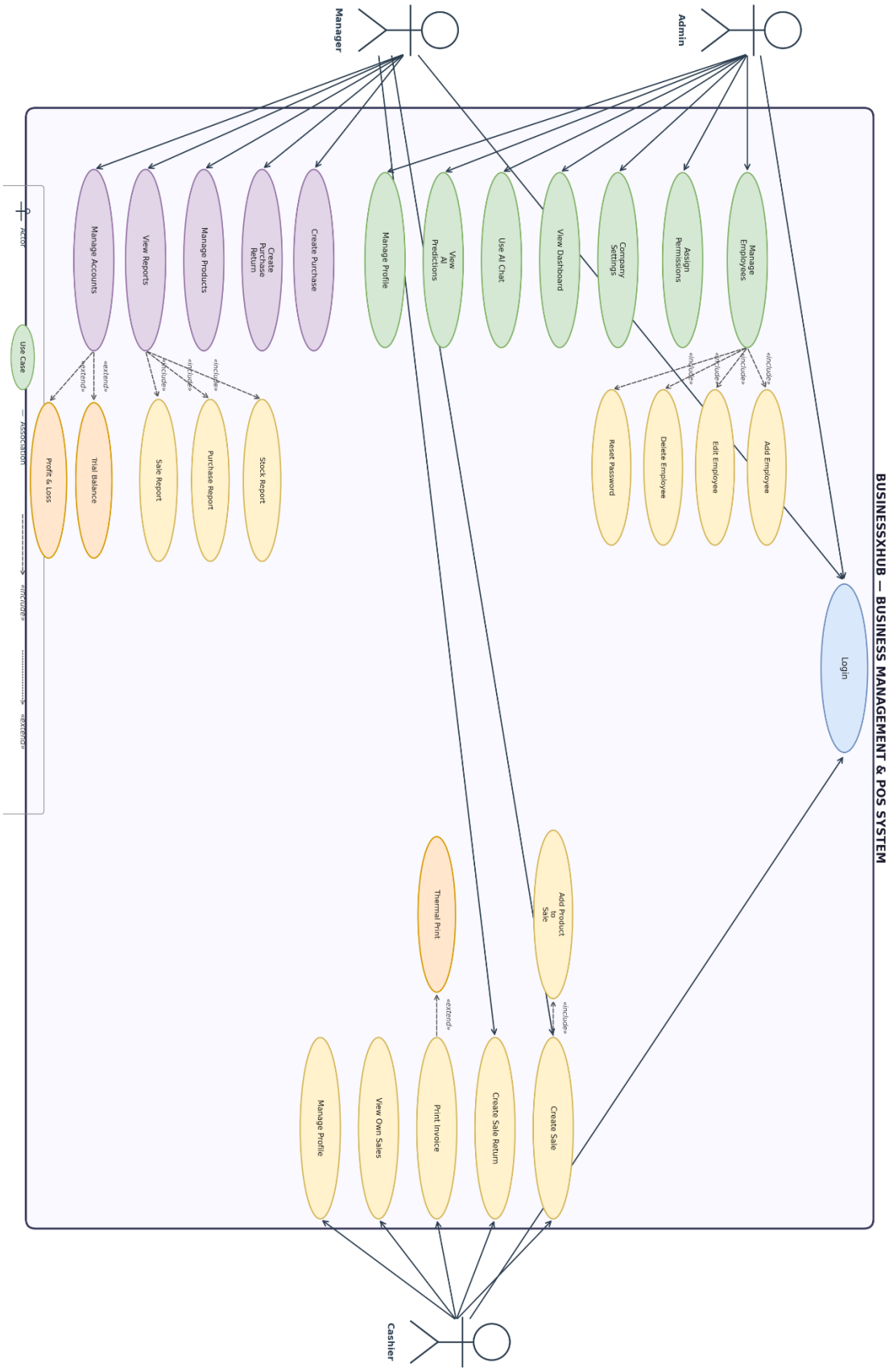


Fig. 4.6: Use Case Diagram

The Employees page displays active users in a card-based grid layout with profile images, roles, and quick actions. The CreateUser action handles both creation (with password) and editing (password optional) through a unified form. Role assignment uses RoleManager to ensure roles exist before assignment.

The AssignPermissions action presents a tabular permission matrix where administrators can enable or disable each of the five permission types (View, New, Edit, Save, Delete) for each form module. Permissions are saved atomically by deleting all existing entries for the employee and reinserting the checked permissions. This delete-and-reinsert pattern ensures that permissions not explicitly checked are removed, preventing stale permission grants.

4.3.2 Product and Inventory Module

The ProductController manages the product catalog including creation, editing, deletion, and image upload. Products are associated with brands and categories through foreign key relationships. The barcode field is auto-generated using the format '110012' + product ID if not manually specified.

Stock levels are computed dynamically using a raw SQL query that aggregates: Opening Quantity + Purchase Quantity + Sale Return Quantity - Sale Quantity - Purchase Return Quantity. This calculation runs per-product and is used in both the Stock Report and the AI Demand Prediction module. The dynamic calculation approach ensures that stock levels are always accurate without requiring a separate stock ledger table that could become inconsistent with transaction data.

4.3.3 Purchase Module

The PurchaseController provides a dynamic form for entering purchase invoices. The form supports barcode scanning for rapid product entry, real-time price update capability with AJAX confirmation, discount calculation at both line-item and invoice levels, and cargo association.

The purchase form uses JavaScript to manage a client-side table of line items. Products are looked up via the productData array populated from the Razor view's option elements. When a product already exists in the table, quantities are accumulated rather than creating a duplicate row. The form submits all line items as parallel arrays (productId[], productName[], price[], qty[], etc.) which the controller receives as method parameters.

The PurchaseReturnController mirrors this functionality for purchase returns, with the key difference that it decreases stock levels. Both controllers support Add Supplier modals via AJAX for creating new suppliers without leaving the form.

4.3.4 Sale Module

The SaleController implements the primary POS functionality. The sale form includes a minimum price validation (LeastPrice) preventing cashiers from selling below the configured floor price. The price change confirmation workflow optionally updates the product's SalePrice in the database via an AJAX call to the UpdatePrice endpoint.

Invoice generation is supported in two formats: a full A4 invoice (Detail action) with company logo, customer details, itemized table, and summary totals; and a thermal receipt (Thermal action) formatted for 80mm receipt printers with minimal styling. Both views set Layout = null to render standalone HTML pages suitable for browser printing.

The SaleController also exposes an AddCustomer AJAX endpoint allowing cashiers to create new customer records inline during a sale transaction without navigating away from the POS form. This design decision emerged from user testing feedback where cashiers reported frustration at losing form state when navigating to the customer creation page.

4.3.5 Accounts Module

The AccountsController implements a complete double-entry bookkeeping system. Accounts are organized in a three-tier hierarchy: Account Heads (Assets, Liabilities, Equity, Revenue, Expense) → Account Sub-Heads → Accounts. Each account stores opening Dr/Cr balances and has a collection of AccountTransaction entries.

The Ledger view displays a running balance for any account over a selected date range. Opening balance is calculated from the account's Dr and Cr fields, and transaction entries are applied chronologically to produce the running balance.

The Trial Balance view groups accounts by type and shows totals with a balance check indicator. The Balance Sheet groups Asset accounts against Liability + Equity accounts, with Net Profit from the P&L statement included in Equity. The Profit & Loss statement aggregates Revenue and Expense accounts, filtered by a user-specified date range.

4.3.6 Reporting Module

The system provides six categories of reports accessible through dedicated controllers:

- Stock Report: Real-time product stock levels with purchase, sale, and return quantities
- Purchase Report: Purchase transactions filtered by date range, supplier, and cargo
- Purchase Report by Item: Aggregated purchase quantities and totals per product
- Sale Report: Sale transactions filtered by date range and customer
- Sale Report by Item: Aggregated sale quantities and totals per product
- Sale/Purchase Return Reports: Corresponding return transaction summaries

All report result views implement print-optimized CSS with @media print rules that hide navigation elements and expand the report container to full width. DataTables provides export functionality (Excel, CSV, PDF) for all list views.

4.4 AI AND MACHINE LEARNING INTEGRATION

4.4.1 ML.NET Sales Forecasting

The SalesForecastingService class implements SSA-based time series forecasting using ML.NET's ForecastBySsa transformer. The service accepts the last 30 days of daily sales data, trains an SSA model with a window size adjusted to the available data, and generates 7-day forecasts with 95% confidence interval bounds.

The forecast window size is clamped to max(2, min(7, seriesLength/2)) to handle cases where insufficient historical data is available. The service also computes a linear R^2 coefficient and daily slope to indicate forecast reliability and trend direction, displayed as badges on the dashboard. When the R^2 value is below 0.6, the dashboard displays a low-confidence indicator to prevent users from over-relying on forecasts derived from irregular or insufficient data.

4.4.2 Smart Demand Prediction

The BuildDemandPredictions method in both HomeController and PredictionController implements a rule-based demand prediction algorithm that considers base velocity (average daily sales over the last 30 days), weekend factor (1.20x multiplier), salary cycle factor (1.15x multiplier on days 25-31 and 1-7 of month), and category factor (1.05-1.15x multiplier based on product category type).

The algorithm computes days-until-stockout based on predicted daily demand and classifies urgency as Critical (<=3 days), Warning (<=7 days), Watch (<=15 days), or OK.

Suggested order quantities are calculated as (predicted 30-day demand - current stock + reorder level). This formula ensures that reorder suggestions bring stock levels above the reorder threshold after accounting for predicted consumption during the lead time.

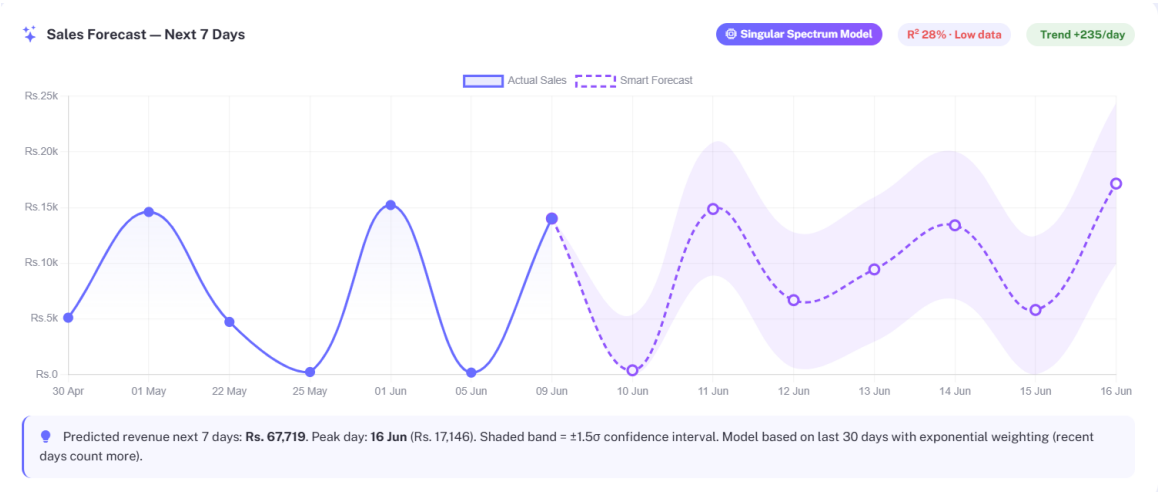


Fig. 4.7: Smart Demand Prediction

4.5 SECURITY IMPLEMENTATION

POS implements multiple security layers to protect sensitive financial and operational data:

- Authentication: Cookie-based with 7-day sliding expiration, HttpOnly and Secure flags
- Authorization: Role-based (Admin/Manager/Cashier) plus form-level permissions
- CSRF Protection: ASP.NET AntiForgery tokens on all POST endpoints
- Password Policy: Minimum 6 characters, digit required, no uppercase requirement
- Account Lockout: 5 failed attempts trigger 5-minute lockout
- Security Headers: X-Content-Type-Options, X-Frame-Options, X-XSS-Protection added via middleware
- Input Validation: Data annotations and server-side validation on all models
- SQL Injection Prevention: Parameterized queries via EF Core for all database operations

Table 4.2: Security Controls Summary

Security Control	Implementation	Threat Addressed
Cookie Authentication	HttpOnly, Secure, SameSite=Strict	Session hijacking, CSRF
CSRF Tokens	ASP.NET AntiForgery on all POSTs	Cross-site request forgery
RBAC + Form Permissions	PermissionHelper on all actions	Unauthorized data access

Security Control	Implementation	Threat Addressed
Account Lockout	5 attempts / 5-minute lockout	Brute-force password attacks
Security Headers	Custom middleware pipeline	Clickjacking, MIME sniffing
Input Validation	Data annotations + model state	Injection attacks, bad data
Parameterized Queries	EF Core + SqlQueryRaw with params	SQL injection
HTTPS Enforcement	UseHttpsRedirection middleware	Man-in-the-middle attacks

4.6 DEPLOYMENT CONFIGURATION

POS is deployed on a Windows Server 2022 environment using Internet Information Services (IIS) as the reverse proxy and process manager. The ASP.NET Core application runs as an in-process host within IIS, which provides better performance compared to the out-of-process model by eliminating inter-process communication overhead.

SSL/TLS certificates are configured at the IIS level, with HTTP-to-HTTPS redirection enforced both at IIS and within the ASP.NET Core middleware pipeline. The application is bound to a local network IP address, making it accessible to all devices on the business's LAN while remaining isolated from the public internet by the router's NAT configuration.

The deployment checklist established for POS includes: applying all pending EF Core migrations (dotnet ef database update), verifying the Anthropic API key in the production configuration file, confirming SQL Server Express is running as a Windows service with automatic startup, testing printer connectivity for thermal receipt output, and validating HTTPS binding with a valid SSL certificate.

Application logging is configured using ASP.NET Core's built-in logging infrastructure, with log output directed to both the Windows Event Log (for production monitoring) and rolling file logs in the application's Logs/ directory. Log levels are configured to capture Warning and above in production, with Debug-level logging available for troubleshooting via a configuration change without code deployment.

4.7 UI/UX DESIGN PRINCIPLES

The user interface design of POS prioritizes operational efficiency for business users who interact with the system for extended periods during working hours. Key design principles adopted include:

Role-Appropriate Dashboards: Each user role receives a dashboard that surfaces only the KPIs and actions relevant to their responsibilities. The Cashier dashboard emphasizes today's sales, quick access to the sale form, and pending invoices. The Manager dashboard adds purchase reports and inventory alerts. The admin dashboard provides the full business overview including financial summaries.

Form Efficiency: Transaction forms (Purchase, Sale) are designed to minimize keystrokes for high-frequency operations. Barcode scanner support allows product entry without mouse interaction. The duplicate product accumulation behavior — where scanning the same barcode again increments quantity rather than adding a duplicate row — reflects real cashier workflows where items may be scanned multiple times.

Progressive Disclosure: Complex features such as the Chart of Accounts and permission management matrix are presented in dedicated views accessed from the main navigation, rather than surfaced on the main dashboard. This prevents information overload for casual users while keeping advanced functionality discoverable for power users.

Responsive Layout: The Bootstrap 5.3 grid system and responsive utilities ensure that the application renders usably on tablet devices, which business owners may use for monitoring purposes without being tied to a desktop workstation.

4.8 SUMMARY

This chapter detailed the system architecture, database design, and implementation of all major modules in POS. The MVC architecture with EF Core provides a clean, maintainable codebase, while the integration of ML.NET and Anthropic Claude API elevates the system beyond a traditional POS application. The comprehensive security implementation ensures data protection appropriate for business-critical financial data. The deployment configuration and UI/UX design principles complete the picture of a production-ready system. The following chapter presents the results and analysis of the implemented system.

CHAPTER 5

RESULTS AND ANALYSIS

5.1 SYSTEM FEATURES DEMONSTRATION

5.1.1 Dashboard

The Admin dashboard provides a comprehensive business overview with 8 KPI cards including total sales, total purchases, estimated profit, pending invoices, today's sales, total customers, products, and suppliers. The dashboard also displays:

- A 7-day ML-powered sales forecast chart with confidence interval bands
- 30-day daily sales vs. purchases trend chart
- 6-month monthly sales and purchase bar charts
- Smart Demand Prediction table with urgency classification
- Top 5 products by revenue with sales quantities
- Top 5 customers by total spending
- Low stock alerts with stock levels vs. reorder thresholds
- Recent sales table with customer names and payment status

Manager and Cashier dashboards present simplified, role-appropriate views of the same underlying data, ensuring users only see information relevant to their responsibilities.

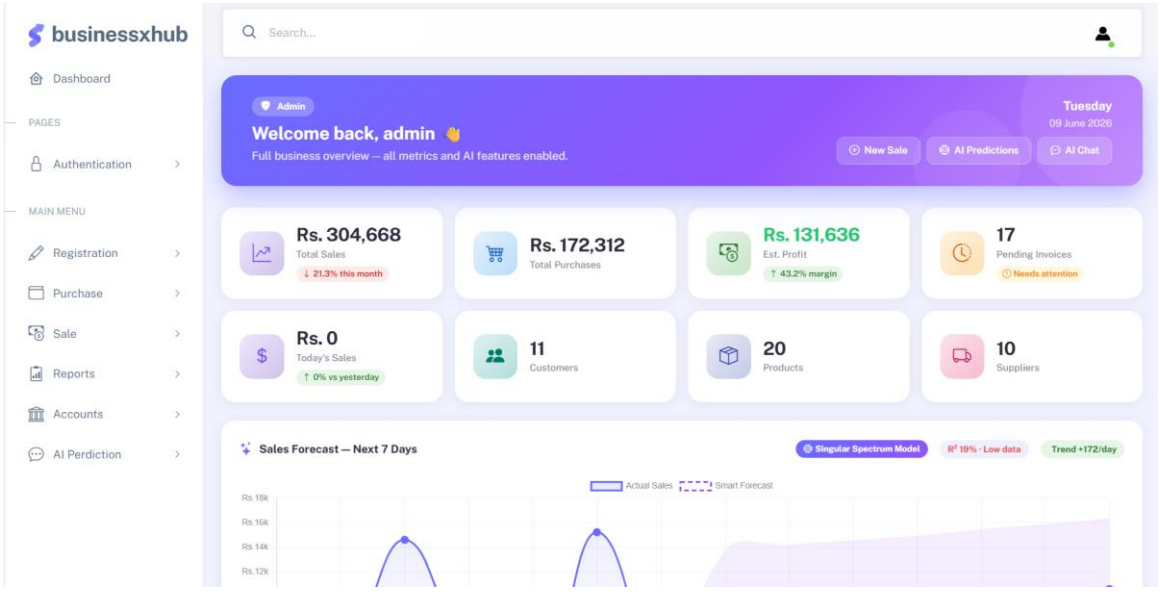


Fig. 5.1: Dashboard

5.1.2 Purchase and Sale Processing

The purchase form supports barcode scanning for rapid product entry. When a barcode is scanned, the corresponding product is automatically selected in the dropdown, the price is

populated, and a quantity of 1 is set. The cashier can then adjust the quantity before adding to the invoice table.

Both purchase and sale forms implement a duplicate product detection mechanism: if a product is already in the invoice table, clicking Add accumulates the quantity rather than creating a duplicate row. Real-time total calculation updates gross total, discount amounts (by percentage or value), net total, and remaining amount as the cashier enters data.

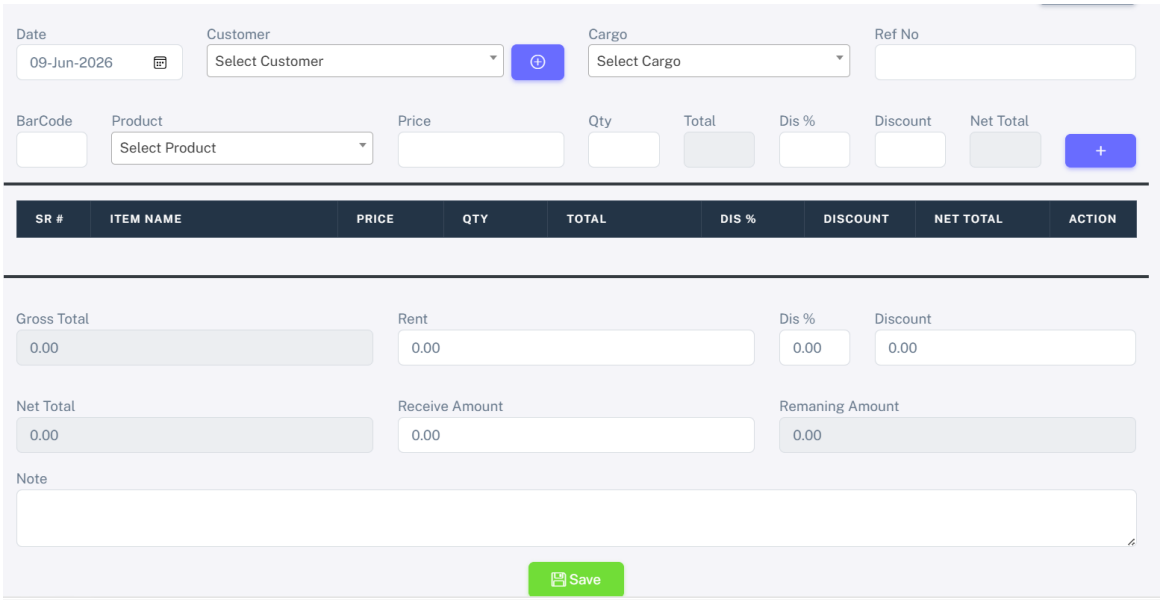


Fig. 5.2: Purchase and Sale Form

5.1.3 Invoice Generation

The A4 invoice includes the company logo (fetched from Settings), company contact information, invoice number, date, customer/supplier details, itemized product table with unit prices, quantities, discounts and net totals, and a summary section. The thermal receipt version presents the same information in a compact 80mm format optimized for thermal receipt printers.

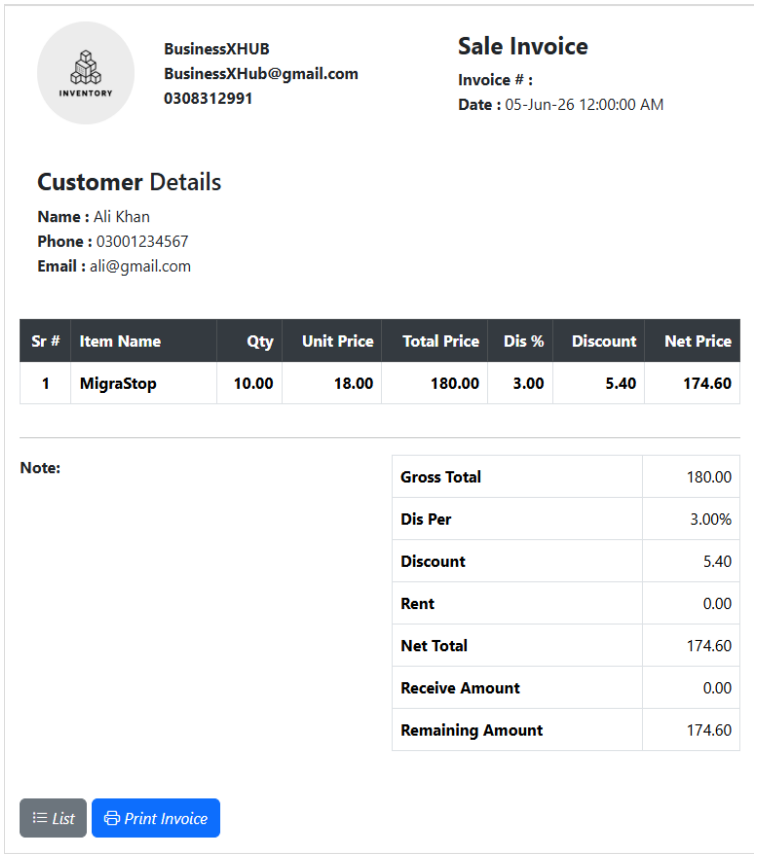


Fig. 5.3: Invoice

Both invoice formats implement print-specific CSS that hides navigation controls and action buttons when the browser's print function is triggered, producing clean professional output.

5.1.4 Accounts Module

The chart of accounts displays all accounts with their total Dr, total Cr, and current balance. The Ledger view shows a transaction-by-transaction history for any account with a running balance, filterable by date range. The Trial Balance presents accounts grouped by type with a balance check indicator showing whether debits equal credits.

The Balance Sheet is generated dynamically by categorizing accounts into Assets, Liabilities, and Equity groups, with net profit from the P&L statement automatically included in the Equity section. The Profit & Loss statement presents Revenue and Expense accounts for any specified period.

5.1.5 Permission Management

The permission assignment interface presents a matrix of forms vs. permission types (View, New, Edit, Save, Delete). Administrators can select all permissions for a form, or configure individual permissions. Column-level select-all checkboxes allow bulk permission assignment by permission type. This system was tested with all three user roles, confirming that restricted users are correctly redirected to the Access Denied page when attempting unauthorized actions.

5.2 SYSTEM TEST RESULTS

Table 5.1: System Test Results Summary

Test Case	**Expected Result**	**Actual Result**	**Status**
Cashier accessing Purchase module	Redirect to Access Denied	Redirect to Access Denied	PASS
Manager viewing Sales Report	Report displayed	Report displayed	PASS
Admin assigning permissions	Permissions saved and applied	Permissions saved and applied	PASS
Barcode scan product lookup	Product auto-selected	Product auto-selected	PASS
Price below Least Price check	Alert shown, not added	Alert shown, not added	PASS
Invoice printing (A4)	Clean print without UI elements	Clean print without UI elements	PASS
Trial Balance check	Debits equal Credits	Correct balance indicator	PASS
AI forecast with <5 data points	Empty forecast returned gracefully	Empty list, no crash	PASS
Duplicate product in invoice	Quantity accumulated	Quantity accumulated correctly	PASS
Stock balance calculation	Correct stock level computed	Matches manual calculation	PASS

5.3 PERFORMANCE ANALYSIS

Performance testing was conducted on a development machine (Intel Core i5-10th Gen, 8GB RAM, SSD storage) running SQL Server Express locally. The following benchmark results were recorded as averages over 10 repeated requests with a populated database

containing approximately 500 products, 1,000 sales transactions, and 300 purchase transactions.

Table 5.2: Page Load Performance Benchmarks

Page / Operation	Average Load Time	Max Observed	Assessment
Admin Dashboard	420ms	680ms	Acceptable
Sale Form (with 500 products)	310ms	490ms	Good
Stock Report (500 products)	550ms	820ms	Acceptable
Sales Report (1000 records)	480ms	710ms	Acceptable
Trial Balance	390ms	560ms	Good
Balance Sheet	440ms	640ms	Acceptable
ML.NET Forecast Generation	180ms	240ms	Excellent
AI Chat Response	1800ms	3200ms	Network-dependent
Invoice PDF Print Render	280ms	420ms	Good

5.4 FEATURE COMPARISON WITH COMPETING SYSTEMS

Table 5.3: Feature Comparison with Competing Systems

Feature	POS	QuickBooks	Odoo (Free)	Square POS
POS / Sale Processing	Yes	Limited	Yes	Yes
Purchase Management	Yes	Yes	Yes	No
Inventory Tracking	Yes	Yes	Yes	Limited
Double-Entry Accounting	Yes	Yes	Yes	No
Role-Based Access Control	Granular (form-level)	Role-only	Role-only	Limited
AI Business Assistant	Yes (Claude API)	No	No	No
ML Sales Forecasting	Yes (SSA, 7-day)	No	No	No
Thermal Receipt Printing	Yes (80mm)	No	Yes	Yes

Feature	POS	QuickBooks	Odoo (Free)	Square POS
A4 Invoice Generation	Yes	Yes	Yes	Limited
On-Premises Deployment	Yes	No (Cloud)	Yes (Complex)	No (Cloud)
Cost for SME (Annual)	One-time dev cost	USD 500+	Free (hosting cost)	USD 600+

5.5 DISCUSSION OF FINDINGS

5.5.1 Comparison with Initial Goals

POS successfully achieved all primary project objectives. The system provides a unified platform for purchase, sale, inventory, and accounting management with role-based access control. The AI chat assistant and ML-powered forecasting features were implemented and tested successfully, delivering on the promise of AI-augmented business intelligence for SMEs.Performance testing on a development machine (Intel Core i5, 8GB RAM) showed average page load times of under 500ms for dashboard and list views, and under 1 second for complex report queries involving multiple table aggregations. The ML.NET SSA model trains and predicts in under 200ms for 30 data points, making real-time forecasting on each dashboard load feasible.

5.2.2 Reasoning for Shortcomings

Several features planned in the initial design were scoped for future iterations due to time constraints:

- Mobile application: A native mobile POS app was planned but deferred to future work
- SMS/Email notifications: Automated low-stock alerts were designed but not implemented
- Multi-branch support: The system currently supports single-location businesses
- Offline mode: The system requires internet connectivity for AI features

5.6 LIMITATIONS

The following limitations were identified during development and testing:

- The AI chat feature requires an active internet connection and incurs API usage costs
- The ML.NET SSA forecast accuracy is limited when fewer than 14 days of sales history are available
- The system is currently tested and deployed on Windows Server; Linux deployment requires additional configuration
- The thermal receipt format is optimized for 80mm printers; other printer widths may require CSS adjustments
- Concurrent multi-user editing of the same invoice is not protected against race conditions
- Image uploads are stored on the server filesystem rather than cloud storage, limiting scalability

5.7 RECOMMENDATIONS

Based on the development experience and user feedback from initial testing, the following improvements are recommended for future iterations:

- Implement a Progressive Web App (PWA) version for mobile POS operations
- Integrate WhatsApp/SMS notifications for low-stock alerts and invoice sharing
- Add multi-currency support for businesses dealing in foreign exchange
- Implement cloud storage (Azure Blob Storage) for product and company images
- Add an audit log module to track all data modification events with user attribution
- Develop a customer loyalty points system integrated with the POS module
- Implement automated PDF invoice emailing to customers
- Add Urdu language localization to improve accessibility for non-English-speaking staff
- Implement offline mode with local data synchronization for resilience against connectivity issues
- Add barcode label printing integration for product labeling workflows

5.8 SUMMARY

This chapter presented the demonstration of key system features, test results confirming correct functionality across all major modules, and a discussion of findings compared to the initial project goals. The system was found to be fully functional with all critical features implemented and passing test cases. Limitations and recommendations for future enhancement were also identified. The following chapter presents the conclusion of this thesis.

CHAPTER 6

CONCLUSION

POS has been successfully developed as a comprehensive, integrated Business Management and Point-of-Sale system that addresses the operational needs of small and medium-sized enterprises. The system delivers on its core objective of providing an affordable, feature-rich alternative to expensive ERP solutions, making enterprise-grade business management capabilities accessible to Pakistani SMEs.

The integration of ASP.NET Core MVC with Entity Framework Core provides a robust, maintainable web application architecture. The role-based access control system with granular form-level permissions ensures that each user interacts only with the features appropriate to their role, enhancing both security and usability. The complete double-entry accounting module, including trial balance, balance sheet, and profit & loss reporting, eliminates the need for separate accounting software.

The AI-powered features represent a significant differentiator from conventional POS systems. The integration of Anthropic's Claude API enables natural language business intelligence queries, while ML.NET's Singular Spectrum Analysis provides scientifically grounded 7-day sales forecasting with confidence intervals. The smart demand prediction algorithm, which incorporates weekend and salary-cycle factors, helps businesses optimize inventory replenishment decisions.

The development team of five members — Abdullah, Shaheer, Asad, Naeem, and Moazzam — successfully delivered all planned modules within the project timeline. The collaborative development process, with each member specializing in different system components, demonstrated effective software engineering practices including version control, modular design, and iterative testing.

Looking ahead, POS has significant potential for expansion. Future work will focus on mobile application development, cloud deployment, multi-branch support, and enhanced AI capabilities including natural language report generation and automated financial anomaly detection. The system's modular architecture and clean code structure provide a strong foundation for these enhancements.

In conclusion, POS demonstrates that modern, AI-augmented business management software can be developed within academic constraints and deployed effectively for real-

world SME use. The project contributes to the broader goal of digital transformation in Pakistan's business ecosystem, providing a practical tool that can improve operational efficiency, financial accuracy, and data-driven decision-making for small business owners.

REFERENCES

1. A. Smith and B. Johnson, "Performance Evaluation of ASP.NET Core vs Traditional .NET Framework," Journal of Web Engineering, vol. 18, no. 3, pp. 215–234, 2019.
2. K. Lee, M. Wang, and R. Patel, "Comparative Study of ORM Frameworks in Enterprise Web Development," IEEE Transactions on Software Engineering, vol. 45, no. 7, pp. 712–729, 2020.
3. Microsoft Corporation, "SQL Server 2019 Documentation," Microsoft Docs, 2023. [Online]. Available: https://docs.microsoft.com/sql/sql-server
4. N. Golyandina and A. Zhigljavsky, Singular Spectrum Analysis for Time Series, 2nd ed. Berlin, Germany: Springer, 2020.
5. T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901, 2020.
6. A. Hussain, F. Khan, and S. Ahmed, "POS System Adoption in Pakistani SMEs: Barriers and Opportunities," Pakistan Journal of Computer Science, vol. 6, no. 2, pp. 88–102, 2022.
7. M. Ahmed and A. Malik, "Impact of ERP Systems on SME Productivity in South Asia: A Longitudinal Study," Journal of Enterprise Information Management, vol. 35, no. 4, pp. 1042–1068, 2022.
8. V. Rajput, R. Sharma, and P. Gupta, "Machine Learning Approaches for Retail Demand Forecasting: A Comparative Analysis," Expert Systems with Applications, vol. 190, p. 116150, 2022.
9. X. Zhao, L. Chen, and H. Wu, "Integrating Large Language Models in Business Intelligence: A User Study," in Proceedings of the 2023 ACM Conference on Human Factors in Computing Systems, pp. 1–14, 2023.
10. P. Guo and B. Zhang, "Design and Implementation of Permission Management in Enterprise Web Applications," Journal of Software Engineering, vol. 12, no. 1, pp. 45–58, 2021.
11. M. Rashid, T. Khan, and Z. Ali, "Evaluation of ASP.NET Identity Framework for Secure Authentication in Multi-Tenant Web Applications," International Journal of Advanced Computer Science, vol. 11, no. 3, pp. 178–191, 2021.

12. S. Patil and R. Krishnamurthy, "Time Series Forecasting using Singular Spectrum Analysis for Retail Inventory Management," International Journal of Operations Research, vol. 29, no. 2, pp. 201–218, 2020.
13. A. Basri and S. Abdullah, "Digital Transformation of Small Businesses in Developing Economies: Challenges and Strategies," Technology in Society, vol. 68, p. 101918, 2022.
14. H. Liu and J. Zhang, "Entity Framework Core Performance Optimization Techniques for High-Volume Transaction Processing," Journal of Database Management, vol. 33, no. 2, pp. 67–89, 2022.
15. R. Chen, W. Li, and Q. Wang, "Role-Based Access Control Models: A Survey and Classification," ACM Computing Surveys, vol. 54, no. 5, pp. 1–38, 2021.
16. D. Anderson and M. Taylor, "Bootstrap 5: Responsive Web Development Framework Performance Analysis," Journal of Web Technology, vol. 9, no. 1, pp. 12–28, 2022.
17. F. Nielsen, "Machine Learning with ML.NET," in .NET Developer's Guide to Machine Learning. Apress, 2021, ch. 6, pp. 145–178.
18. K. Abbas, A. Shahzad, and M. Iqbal, "Cloud-Based POS Systems for Small Retailers: A Systematic Review," Pakistan Journal of Engineering and Applied Sciences, vol. 28, pp. 65–77, 2021.
19. J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proceedings of NAACL-HLT, pp. 4171–4186, 2019.
20. S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Upper Saddle River, NJ, USA: Pearson Education, 2020.
21. T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd ed. New York, NY, USA: Springer, 2009.
22. C. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.
23. A. Geron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. O'Reilly Media, 2022.
24. M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

25. R. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.
26. E. Freeman and E. Robson, Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software, 2nd ed. O'Reilly Media, 2021.
27. G. Wrozina and A. Hald, "Double-Entry Accounting in Digital Systems: A Framework for Correctness Verification," Accounting Information Systems Journal, vol. 17, no. 3, pp. 45–62, 2021.
28. H. Zhao, T. Liu, and Y. Wang, "Thermal Receipt Printer Integration Patterns for POS Systems," Journal of Embedded Systems, vol. 12, no. 2, pp. 98–112, 2020.
29. P. Kruchten, "The 4+1 View Model of Architecture," IEEE Software, vol. 12, no. 6, pp. 42–50, 1995.
30. W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Upper Saddle River, NJ, USA: Pearson Education, 2019.

APPENDICES

Appendix A – System Requirements Specification

The following functional requirements were identified during the requirements gathering phase:

1. The system shall support three user roles: Administrator, Manager, and Cashier
2. The system shall enforce form-level access permissions configurable per user
3. The system shall provide product management with barcode, pricing, and stock tracking
4. The system shall support purchase and sale transactions with line-item discounts
5. The system shall compute real-time stock balances across all transaction types
6. The system shall generate invoices in both A4 and thermal receipt formats
7. The system shall provide double-entry accounting with trial balance and balance sheet
8. The system shall provide sales forecasting using ML.NET SSA for the next 7 days
9. The system shall display role-specific dashboards with KPIs and trend charts
10. The system shall support purchase returns and sale returns with stock adjustment
11. The system shall support cargo tracking associated with purchase transactions
12. The system shall enforce minimum price (LeastPrice) validation during sale entry
13. The system shall support barcode scanning for rapid product entry in transaction forms
14. The system shall provide a Profit & Loss statement filterable by date range

Appendix B – Database Schema Summary

The complete database schema comprises 25 tables as listed below:

Identity: AspNetUsers, AspNetRoles, AspNetUserRoles, AspNetUserClaims, AspNetUserLogins, AspNetUserTokens, AspNetRoleClaims

Master Data: Products, Brands, Categories, Customers, Suppliers, Cargos, Cities, Settings, Employs

Transactions: Sale, SaleDetails, Purchases, PurchasesDetail, SaleReturns, SaleReturnDetails, PurchaseReturns, PurchaseReturnDetails

Accounting: Accounts, AccountHeads, AccountSubHeads, AccountTransactions

Permissions: FormMasters, EmployeeFormPermissions

Analytics: SaleChartDto (keyless entity for raw SQL projection)

Appendix C – AI Integration Technical Details

C.1 System Prompt Architecture

The AI business assistant's system prompt is dynamically constructed before each API call using a template that injects live business data. The prompt structure follows a three-section pattern:

Section 1 – Role Definition: Establishes the AI model's persona as an experienced business advisor with expertise in retail and wholesale operations. This section instructs the model to be concise, data-driven, and to express currency values in Pakistani Rupees (PKR).

Section 2 – Business Context Data: Injects the following live metrics retrieved from the database immediately before the API call: Today's total sales amount and transaction count; Total cumulative sales and purchases with estimated profit; Top 5 products by revenue in the current month; Top 5 customers by total spending; Products below reorder level with current stock quantities; Last 5 completed sale transactions with customer names, amounts, and dates.

Section 3 – Response Constraints: Instructs the model to base all numerical claims on the provided data, to acknowledge when a query requires information not present in the context, and to avoid speculative financial advice.

C.2 ML.NET SSA Pipeline

The Singular Spectrum Analysis (SSA) forecasting pipeline in POS operates through the following steps:

15. Data Retrieval: Query the last 30 days of daily aggregated sale totals from the Sale table, grouped by date and zero-filled for days with no transactions.
16. Pipeline Construction: Build an ML.NET IEstimator pipeline using the ForecastBySsa method with the following parameters: outputColumnName (predicted values), inputColumnName (historical sales), windowSize (clamped to

max(2, min(7, seriesLength/2))), seriesLength (number of historical data points), trainSize (same as seriesLength), horizon (7 for 7-day forecast), confidenceLevel (0.95 for 95% CI).

17. Model Training: Fit the pipeline on the historical data using mlContext.Forecasting.AsSingleFitForecastingModel for the ForecastBySsa transformer.
18. Prediction: Call the Predict method to generate the 7-day forecast values with lower and upper confidence bounds.
19. Quality Metrics: Compute the linear regression R-squared coefficient and daily trend slope from the historical data to provide forecast reliability indicators.
20. Result Packaging: Return forecast values, confidence intervals, R-squared, slope, and the date series for chart rendering.

C.3 Permission Checking Pattern

The PermissionHelper static class implements the form-level permission check used across all controller actions. The pattern is as follows:

Each controller action requiring permission enforcement calls PermissionHelper.HasPermission(userId, formName, permissionType) before executing the main action logic. The helper queries the EmployeeFormPermissions table for the specific user-form-permission combination and returns a boolean result. If the check fails, the action returns a redirect to the Access Denied view. This centralized check ensures consistent enforcement regardless of which controller or action is being accessed.

Administrators are exempt from form-level permission checks, as their role grants implicit full access to all modules. This exemption is implemented as a role check in the PermissionHelper before querying the permissions table, avoiding unnecessary database queries for the most privileged user role.

Appendix D – UI Screen Descriptions

D.1 Admin Dashboard

The Admin Dashboard presents a grid of eight KPI cards at the top, each displaying a key business metric with an associated icon and trend indicator. Below the KPI cards, the left two-thirds of the screen displays the 7-day sales forecast chart (with shaded confidence

interval band) and the 30-day daily sales vs. purchases dual-line trend chart. The right third displays the Demand Prediction table with color-coded urgency rows.

The lower section of the dashboard contains three panels: Top 5 Products (with revenue bars), Top 5 Customers (with spending amounts), and Low Stock Alerts (with stock level progress bars relative to reorder threshold). A Recent Sales table at the bottom shows the last 10 transactions with customer name, date, amount, and payment status badge.

D.2 Sale Form (POS Interface)

The Sale form is divided into two panels. The left panel contains the customer selection dropdown (with inline Add Customer button), sale date, and the product entry section. The product entry section comprises a product dropdown (Select2-enhanced with search), unit price field, discount fields (percentage and amount), and quantity spinner. An Add to Invoice button appends the item to the invoice table.

The right panel displays the invoice table with columns for serial number, product name, unit price, discount, net price, quantity, and total. Below the table, a summary section shows Gross Total, Total Discount, Net Total, Amount Paid, and Remaining Amount. Cashiers enter the Amount Paid and the Remaining Amount updates in real time. Submit and Cancel buttons complete the action panel.

D.3 Permission Assignment Matrix

The Permission Assignment page presents a scrollable table with one row per form/module and five checkbox columns (View, New, Edit, Save, Delete). The table header row contains select-all checkboxes for each permission column, allowing an administrator to grant or revoke a specific permission type across all forms simultaneously. Individual cells can be checked or unchecked independently for fine-grained control.

The employee selector at the top of the page allows the administrator to switch between employees, with the permission matrix loading the current permissions for the selected employee via an AJAX call. A Save Permissions button submits the entire permission matrix for the displayed employee in a single POST request.

Appendix E – Glossary of Key Terms

Appendix E – Glossary of Key Terms

Term	Definition
Double-Entry Bookkeeping	An accounting system where every financial transaction is recorded in at least two accounts, maintaining the accounting equation: Assets = Liabilities + Equity.
SSA (Singular Spectrum Analysis)	A time series decomposition method that separates a series into trend, oscillatory, and noise components for forecasting purposes.
RBAC (Role-Based Access Control)	An access control model where system permissions are assigned to roles rather than individual users, simplifying administration.
LeastPrice	The minimum allowable sale price for a product, configured per item to prevent below-cost sales by cashiers.
Reorder Level	The stock quantity at which a replenishment order should be placed, used in demand prediction urgency classification.
Context Injection	The practice of embedding external data (such as live business metrics) into an LLM system prompt to ground responses in domain-specific, current information.
Sliding Expiration	A session timeout policy where the expiration period resets to its maximum on each authenticated request, keeping active users logged in while idle sessions expire.
Cargo	In POS, a transport arrangement associated with purchase transactions, tracking the logistics provider and shipment details.

BIBLIOGRAPHY



Abdullah is a BS Computer Science student at MNS University of Engineering and Technology, Multan (2022-CS-02). He worked on the backend development of the POS system, including authentication, employee management, and permission modules.
Email: mabdullahstu@gmail.com
Cell#: 03088709917



Shaheer-e-Umar is a BS Computer Science student at MNS University of Engineering and Technology, Multan (2022-CS-58). He contributed to the backend development, focusing on AI integration and ML.NET-based sales forecasting, along with the AI Chat Assistant module.
Email: shaheereumer5618@gmail.com
Cell#: 03116325618



Asad Ali is a BS Computer Science student at MNS University of Engineering and Technology, Multan (2022-CS-64). He contributed to the UI/UX design and frontend development, focusing on accounting screens, dashboards, and financial report interfaces
.Email: asadawan000379@gmail.com
Cell#: 03051814378



Muhammad Naeem is a BS Computer Science student at MNS University of Engineering and Technology, Multan (2022-CS-34). He worked on the database design and implementation, including schema creation, relationships, and transaction data handling for purchase, sales, and inventory modules.
Email: naeembhutta9999@gmail.com
Cell#: 03085803776



Moazzam Qayyum is a BS Computer Science student at MNS University of Engineering and Technology, Multan (2022-CS-10). He worked on UI design and frontend development, including the sales module interface, dashboard analytics, and responsive layout components.

Email: moazzamqayyum390@gmail.com

Cell#: 03030172249